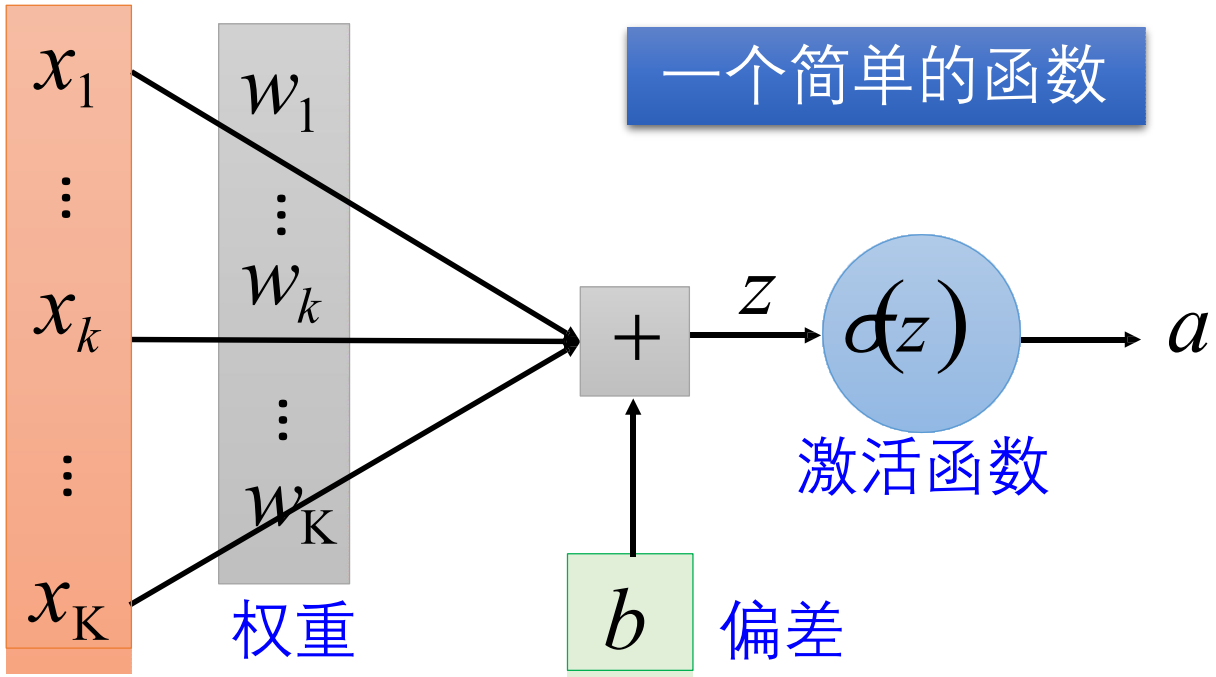
回顾：神经网络

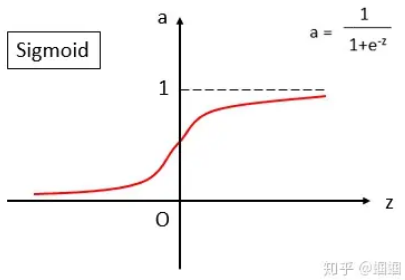
神经元

$$z = x_1 w_1 + \cdots + x_k w_k + \cdots + x_K w_K + b$$

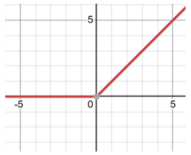


回顾：神经网络

- 激活函数

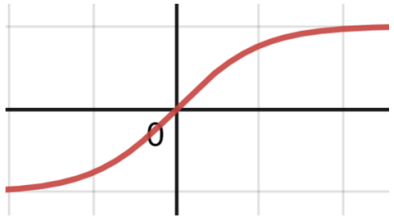


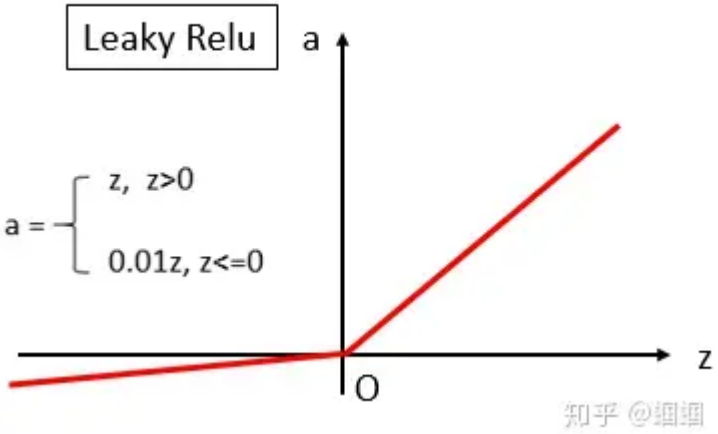
- **ReLU函数**
  - a=max(z,0)
  - 输出为正输入时与输入相等，否则为 0。



- **tanh函数**
  - 输出范围在 -1 到 1 之间

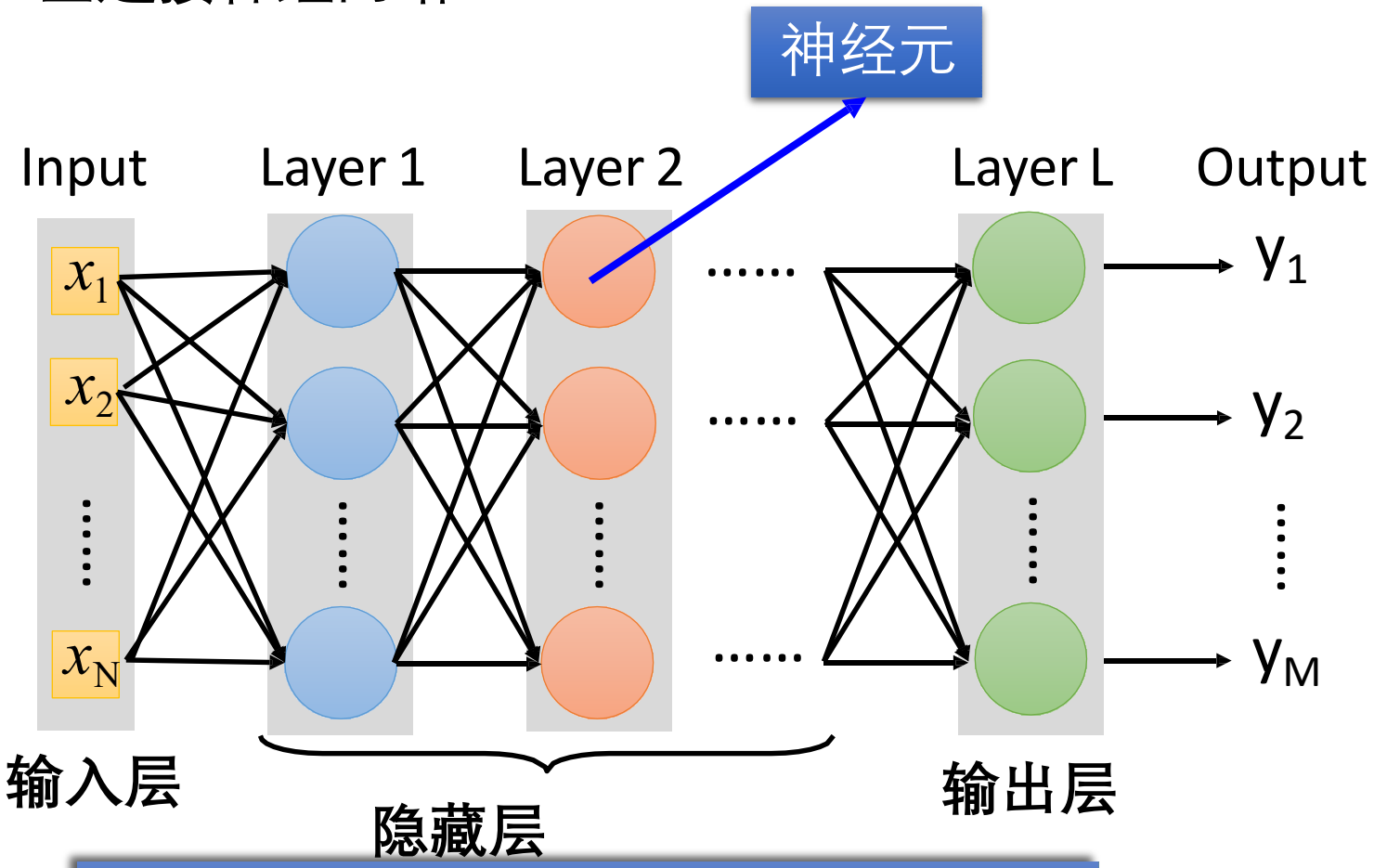
$$a = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$





回顾：神经网络

- 全连接神经网络



深度意味着很多隐藏层

回顾：神经网络

损失

一个好的函数尽可能使损失最低

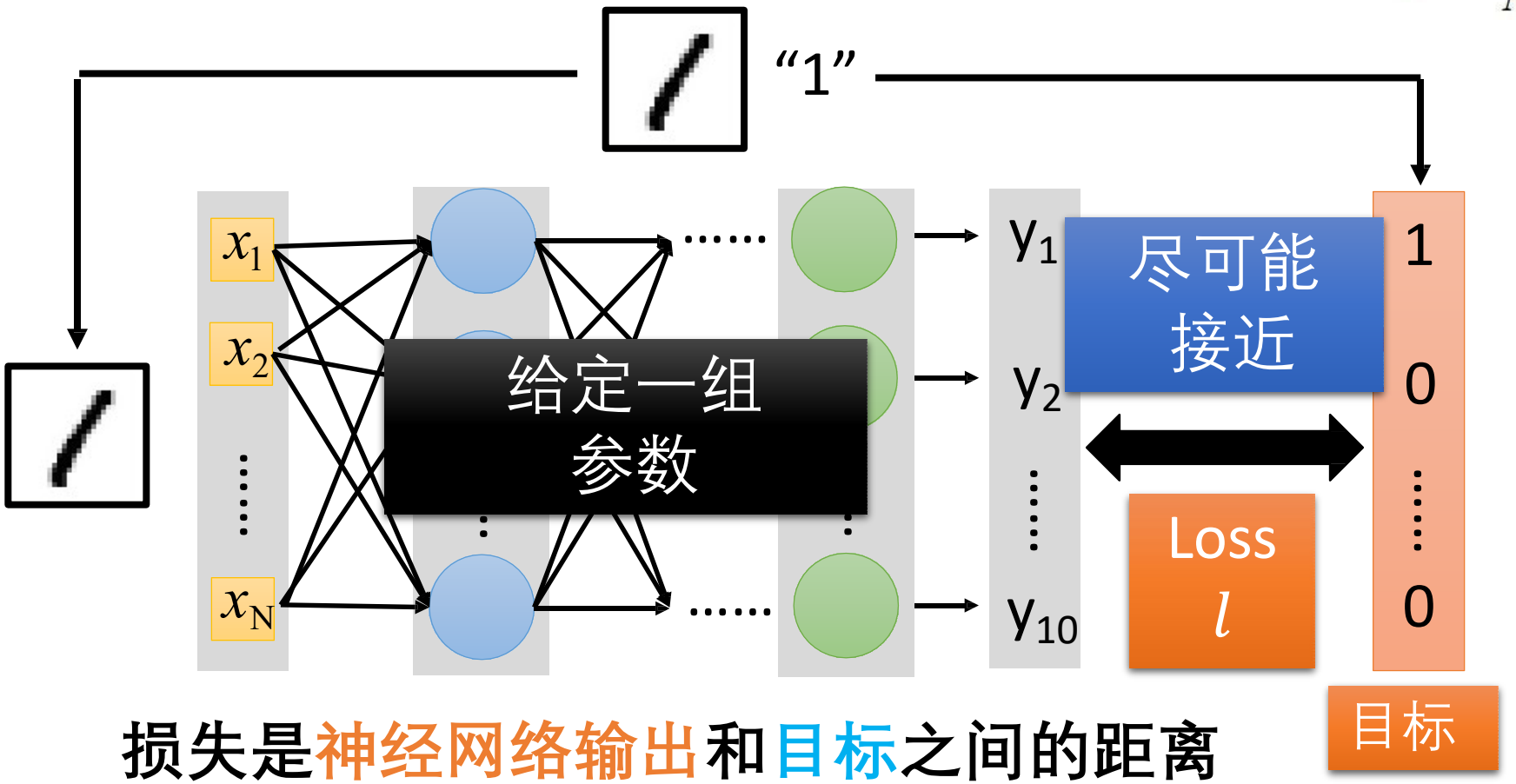
Softmax

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

SVM

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i + R(W)$$ Full loss

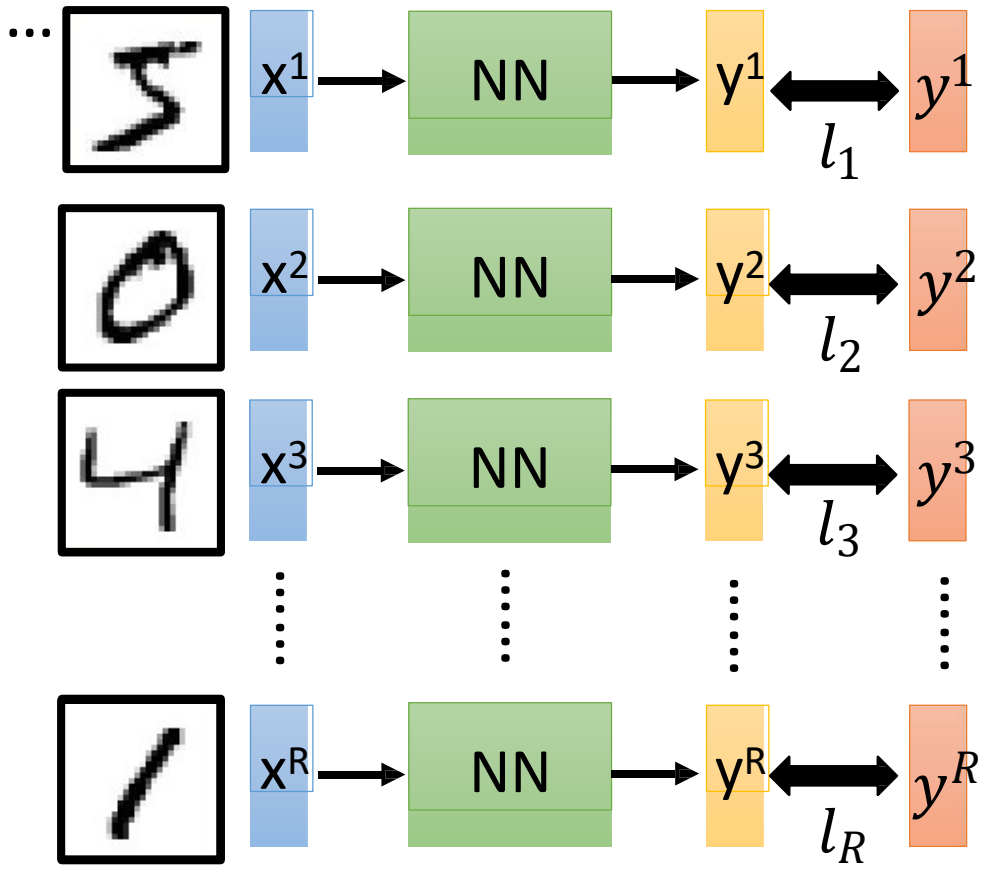


损失是神经网络输出和目标之间的距离

回顾：神经网络

全部损失

对**所有的训练数据**



全部损失:

$$L = \sum_{r=1}^{R} l_r$$

尽可能的小

找到一个函数使损失函数 L 最小

找到参数 $\boldsymbol{\theta}$ 使损失函数 L 最小

最优化

This image is CC0 1.0 public domain

Walking man image is CC0 1.0 public domain

Strategy #1:第一个非常糟糕的解决方案是:随机搜索

```
# assume X_train is the data where each column is an example (e.g. 3073 x 50,000)
# assume Y_train are the labels (e.g. 1D array of 50,000)
# assume the function L evaluates the loss function

bestloss = float("inf") # Python assigns the highest possible float value
for num in xrange(1000):
  W = np.random.randn(10, 3073) * 0.0001 # generate random parameters
  loss = L(X_train, Y_train, W) # get the loss over the entire training set
  if loss < bestloss: # keep track of the best solution
    bestloss = loss
    bestW = W
  print 'in attempt %d the loss was %f, best %f' % (num, loss, bestloss)

# prints:
# in attempt 0 the loss was 9.401632, best 9.401632
# in attempt 1 the loss was 8.959668, best 8.959668
# in attempt 2 the loss was 9.044034, best 8.959668
# in attempt 3 the loss was 9.278948, best 8.959668
# in attempt 4 the loss was 8.857370, best 8.857370
# in attempt 5 the loss was 8.943151, best 8.857370
# in attempt 6 the loss was 8.605604, best 8.605604
# ... (trunctated: continues for 1000 lines)
```

让我们看看这在测试集上的效果如何……

```
# Assume X_test is [3073 x 10000], Y_test [10000 x 1]
scores = Wbest.dot(Xte_cols) # 10 x 10000, the class scores for all test examples
# find the index with max score in each column (the predicted class)
Yte_predict = np.argmax(scores, axis = 0)
# and calculate accuracy (fraction of predictions that are correct)
np.mean(Yte_predict == Yte)
# returns 0.1555
```

15.5% 的准确率! not bad!
(SOTA is ~95%)

Strategy #2:沿着斜坡走

Strategy #2:沿着斜坡走

在一维中，一个函数的导数

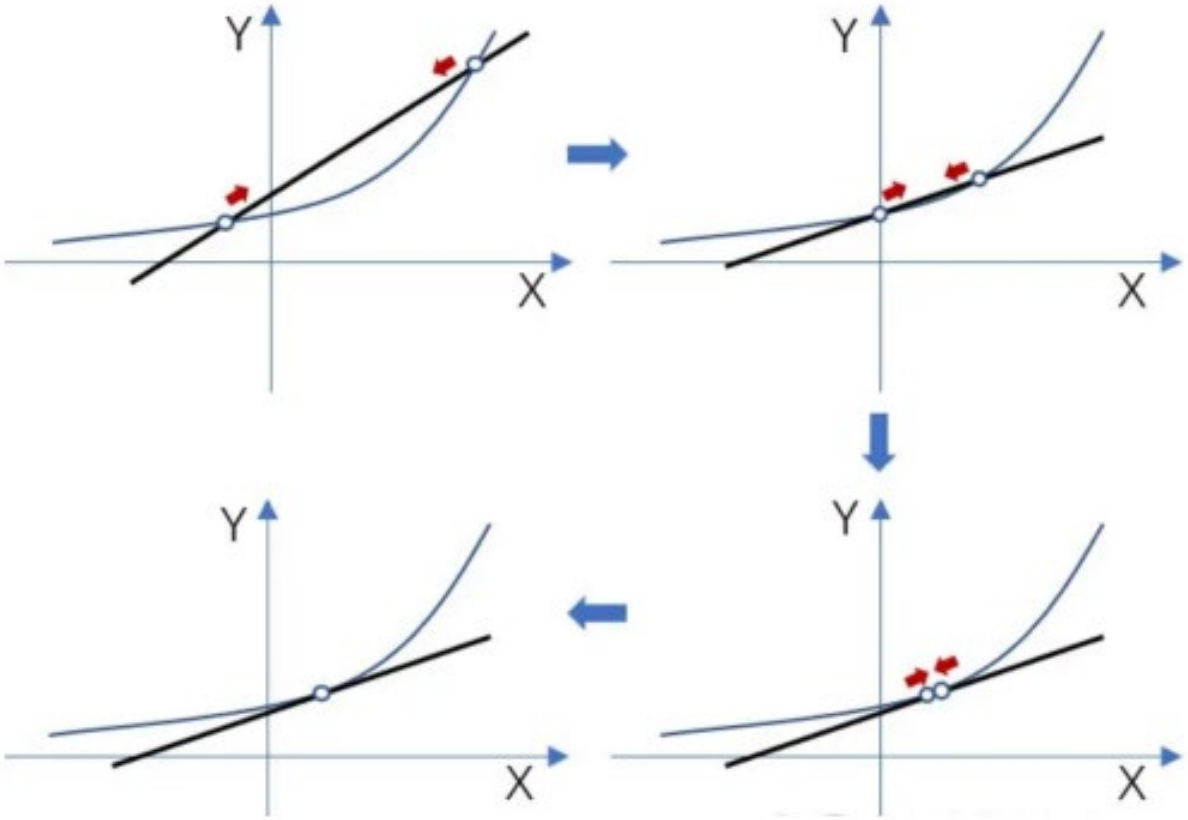
$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

在多维度中，**梯度**是沿每个维度的向量(偏导数)
任何方向的斜率是该方向与梯度的点积，下降最陡的方向是负梯度

导数

- 表示某个瞬间的变化量
- 导数的方向是函数在该点上**变化最快**的方向

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$



导数

- 表示某个瞬间的变化量

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

- 基本导数公式

$(C)' = 0$,

$(x^{\mu})' = \mu x^{\mu-1}$,

$(\sqrt{x})' = \frac{1}{2\sqrt{x}}$, $(\frac{1}{x})' = -\frac{1}{x^2}$,

$(\sin x)' = \cos x$,

$(\cos x)' = -\sin x$,

$(\tan x)' = \sec^2 x$,

$(\cot x)' = -\csc^2 x$,

$(\sec x)' = \sec x \tan x$,

$(\csc x)' = -\csc x \cot x$,

$(a^x)' = a^x \ln a$, $(e^x)' = e^x$,

$(\log_a x)' = \frac{1}{x \ln a}$, $(\ln x)' = \frac{1}{x}$,

$(\arcsin x)' = \frac{1}{\sqrt{1-x^2}}$,

$(\arccos x)' = -\frac{1}{\sqrt{1-x^2}}$,

$(\arctan x)' = \frac{1}{1+x^2}$,

$(\operatorname{arccot} x)' = -\frac{1}{1+x^2}$.

导数

- 表示某个瞬间的变化量

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

- 基本导数公式

$(C)' = 0$,

$(x^{\mu})' = \mu x^{\mu-1}$,

$(\sqrt{x})' = \frac{1}{2\sqrt{x}}$, $(\frac{1}{x})' = -\frac{1}{x^2}$,

$(\sin x)' = \cos x$,

$(\cos x)' = -\sin x$,

$(\tan x)' = \sec^2 x$,

$(\cot x)' = -\csc^2 x$,

$(\sec x)' = \sec x \tan x$,

$(\csc x)' = -\csc x \cot x$,

$(a^x)' = a^x \ln a$, $(e^x)' = e^x$,

$(\log_a x)' = \frac{1}{x \ln a}$, $(\ln x)' = \frac{1}{x}$,

$(\arcsin x)' = \frac{1}{\sqrt{1-x^2}}$,

$(\arccos x)' = -\frac{1}{\sqrt{1-x^2}}$,

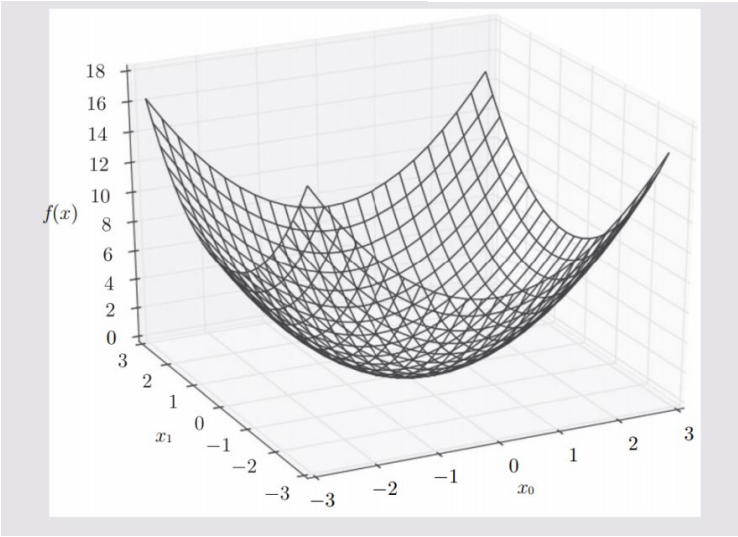
$(\arctan x)' = \frac{1}{1+x^2}$,

$(\operatorname{arccot} x)' = -\frac{1}{1+x^2}$.

偏导数

- 有多个变量的函数的导数称为偏导数。
- 当求某个变量的导数时，就是把其它变量视为**常量**，然后整个函数求其导数

$$\frac{\partial f}{\partial x_0}、\frac{\partial f}{\partial x_1}$$



$$f(x_0, x_1) = x_0^2 + x_1^2$$

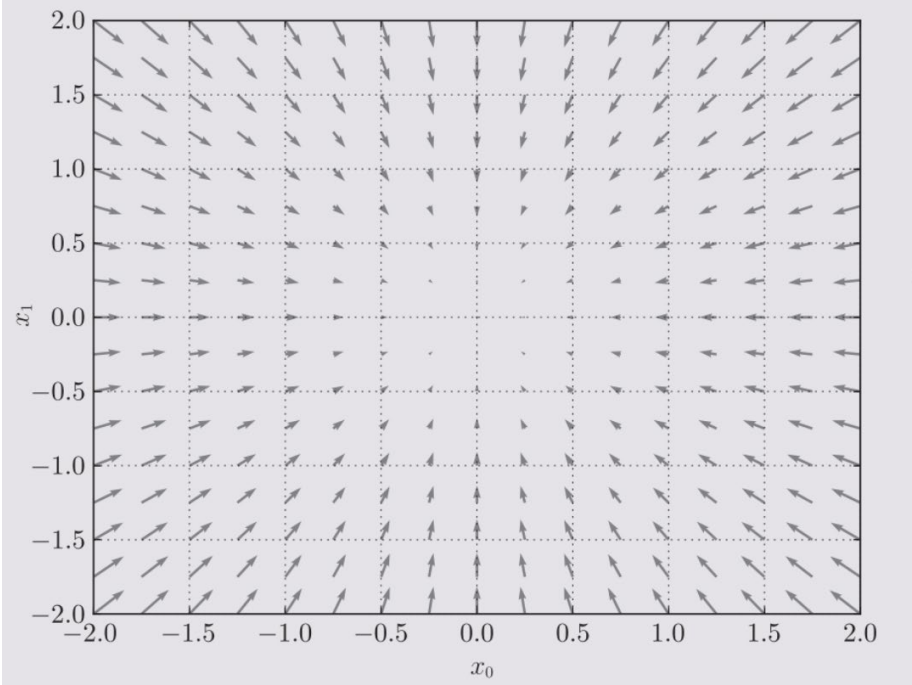
https://colab.research.google.com/drive/1YrZ-cvZHWrxOHbc_F29s6vTolSnxFpgF?usp=sharing

梯度

- 由全部变量的偏导数汇总而成的向量称为梯度（gradient）

$$\left(\frac{\partial f}{\partial x_0}, \frac{\partial f}{\partial x_1}\right)$$

- 梯度的负方向是各点处的函数值减小最多的方向。



$f(x_0, x_1) = x_0^2 + x_1^2$ 的梯度

https://colab.research.google.com/drive/1YrZ-cvZHWrxOHbc_F29s6vTolSnxFpgF?usp=sharing

当前 W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25347

梯度 dW:

[?,
?,
?,
?,
?,
?,
?,
?,
?,…]

当前 W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25347

W + h (first dim):

[0.34 + **0.0001**,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25322

梯度 dW:

[?,
?,
?,
?,
?,
?,
?,
?,
?,…]

当前 **W:**

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25347

W + h (first dim):

[0.34 + **0.0001**,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25322

梯度dW:

[**-2.5**,
?,
?,

(1.25322 - 1.25347)/0.0001
= -2.5

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

?,
?,…]

当前 **W:**	**W + h** (second dim):	**梯度dW:**
[0.34,	[0.34,	[-2.5,
-1.11,	-1.11 + **0.0001**,	?,
0.78,	0.78,	?,
0.12,	0.12,	?,
0.55,	0.55,	?,
2.81,	2.81,	?,
-3.1,	-3.1,	?,
-1.5,	-1.5,	?,
0.33,…]	0.33,…]	?,…]
loss 1.25347	**loss 1.25353**	

当前 W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25347

W + h (second dim):

[0.34,
-1.11 + **0.0001**,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25353

梯度dW:

[-2.5,
0.6,
?,
?,

(1.25353 - 1.25347)/0.0001
= 0.6

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

?,…]

当前 **W:**

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25347

W + h (third dim)**:**

[0.34,
-1.11,
0.78 + **0.0001**,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25347

梯度dW:

[-2.5,
0.6,
?,
?,
?,
?,
?,
?,
?,…]

当前 W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25347

W + h (third dim):

[0.34,
-1.11,
0.78 + **0.0001**,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25347

梯度dW:

[-2.5,
0.6,
0,
?,
?,…]

(1.25347 - 1.25347)/0.0001
= 0

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

当前 **W:**

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25347

W + h (third dim):

[0.34,
-1.11,
0.78 + **0.0001**,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25347

梯度dW:

[-2.5,
0.6,
0,
?,
?,
?,…]

数值梯度

- 慢!需要循环遍历所有维度
- 近似值

这很愚蠢。损失只是W的函数:

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i + \sum_k W_k^2$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$s = f(x; W) = Wx$$

want $\nabla_W L$

这很愚蠢。损失只是W的函数:

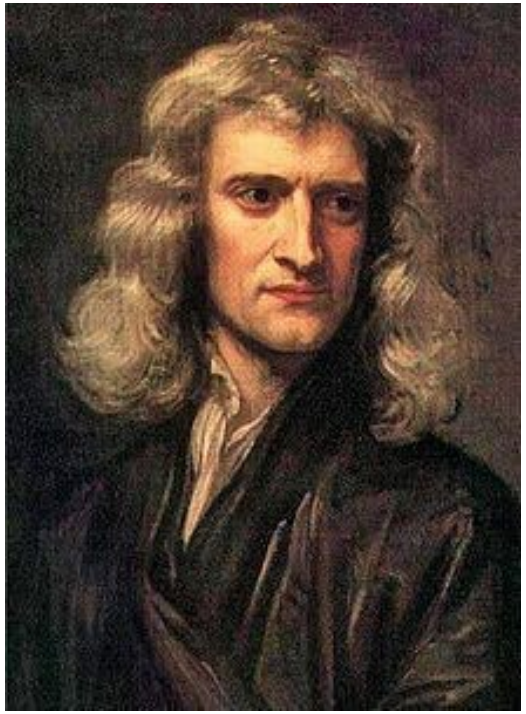
$$L = \frac{1}{N}\sum_{i=1}^{N} L_i + \sum_k W_k^2$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$s = f(x; W) = Wx$$

want $\nabla_W L$

使用微积分计算**解析解**



This image is in the public domain

This image is in the public domain

当前 W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,…]

loss 1.25347

dW = ...
(some function
data and W)

梯度 dW:

[-2.5,
0.6,
0,
0.2,
0.7,
-0.5,
1.1,
1.3,
-2.1,…]

总结:

- 数值梯度:近似，缓慢，易于编写
- 解析梯度:精确，快速，容易受错误影响

=>

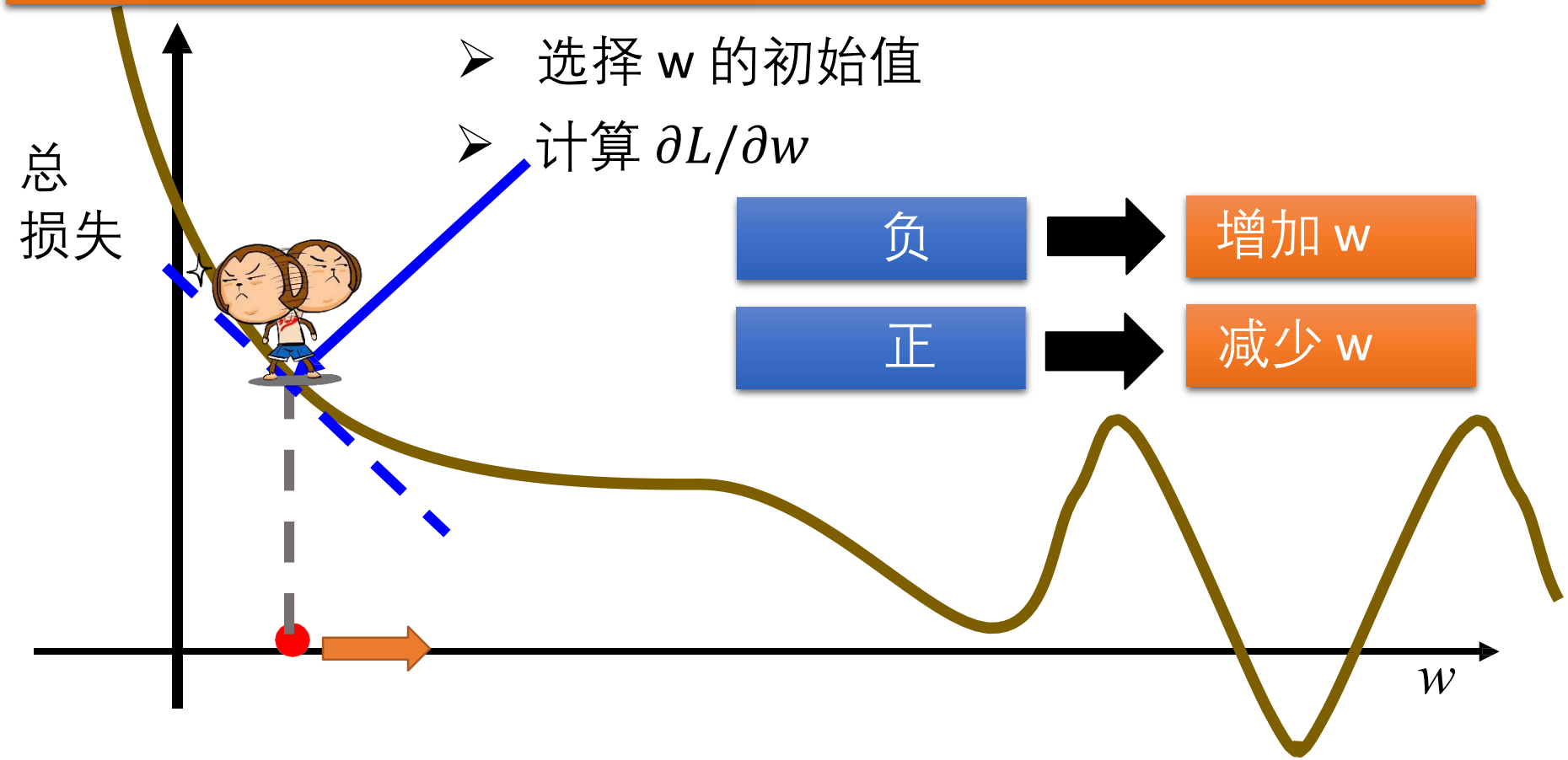
在实践中:总是使用解析梯度，但是用数值梯度检查实现。这被称为梯度检查。

梯度下降

参数$\theta = \{w_1, w_2, \cdots, b_1, b_2, \cdots\}$

找到使总损失最小的参数$\boldsymbol{\theta}^*$

- 选择 w 的初始值
- 计算 $\partial L/\partial w$



负 → 增加 w

正 → 减少 w

梯度下降

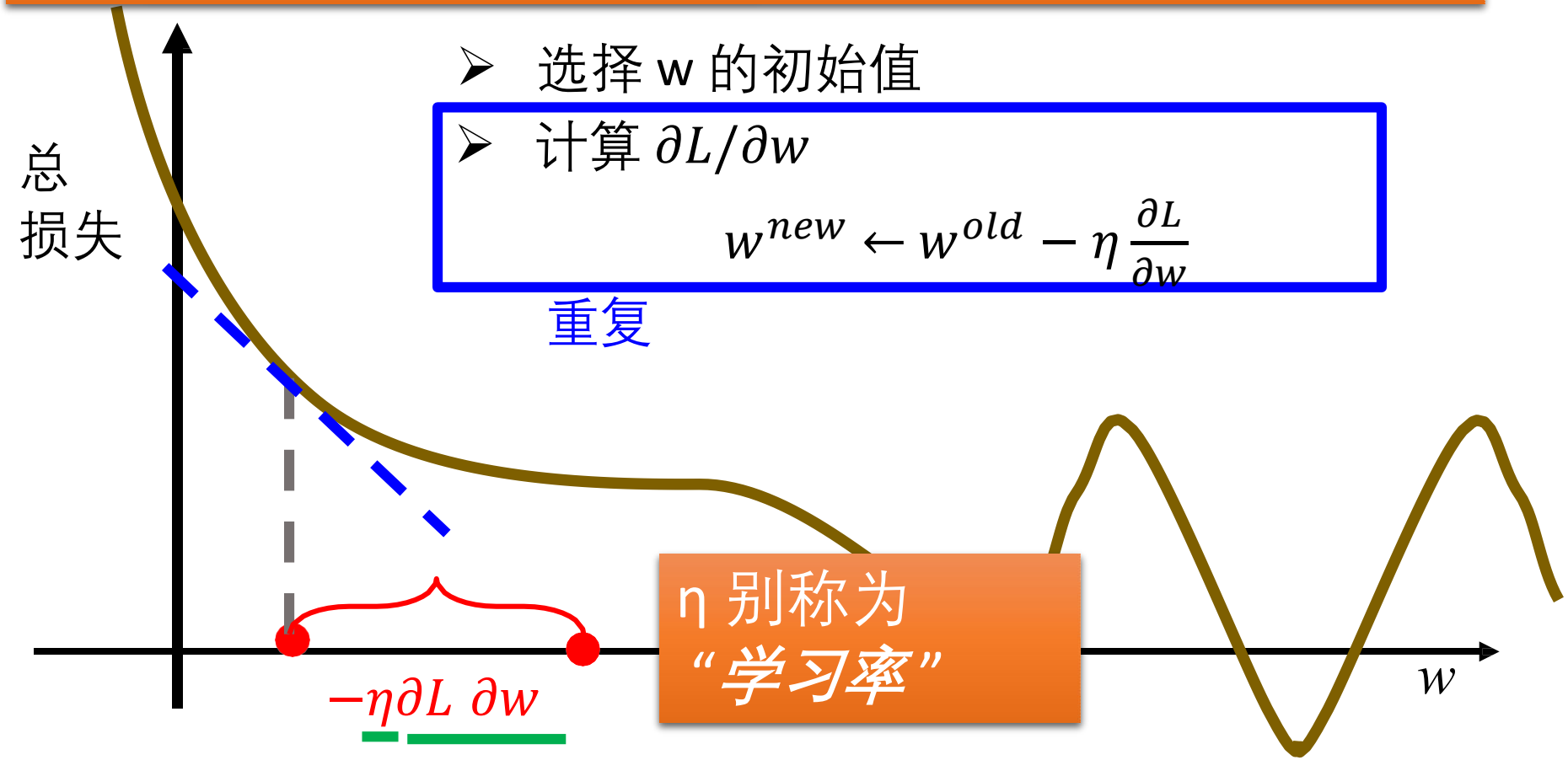
参数 $\theta = \{w_1, w_2, \cdots, b_1, b_2, \cdots\}$

找到使总损失最小的参数$\boldsymbol{\theta}^*$

- 选择 w 的初始值
- 计算 $\partial L/\partial w$

$$w^{new} \leftarrow w^{old} - \eta \frac{\partial L}{\partial w}$$

重复



η 别称为
"学习率"

梯度下降

参数$\theta = \{w_1, w_2, \cdots, b_1, b_2, \cdots\}$

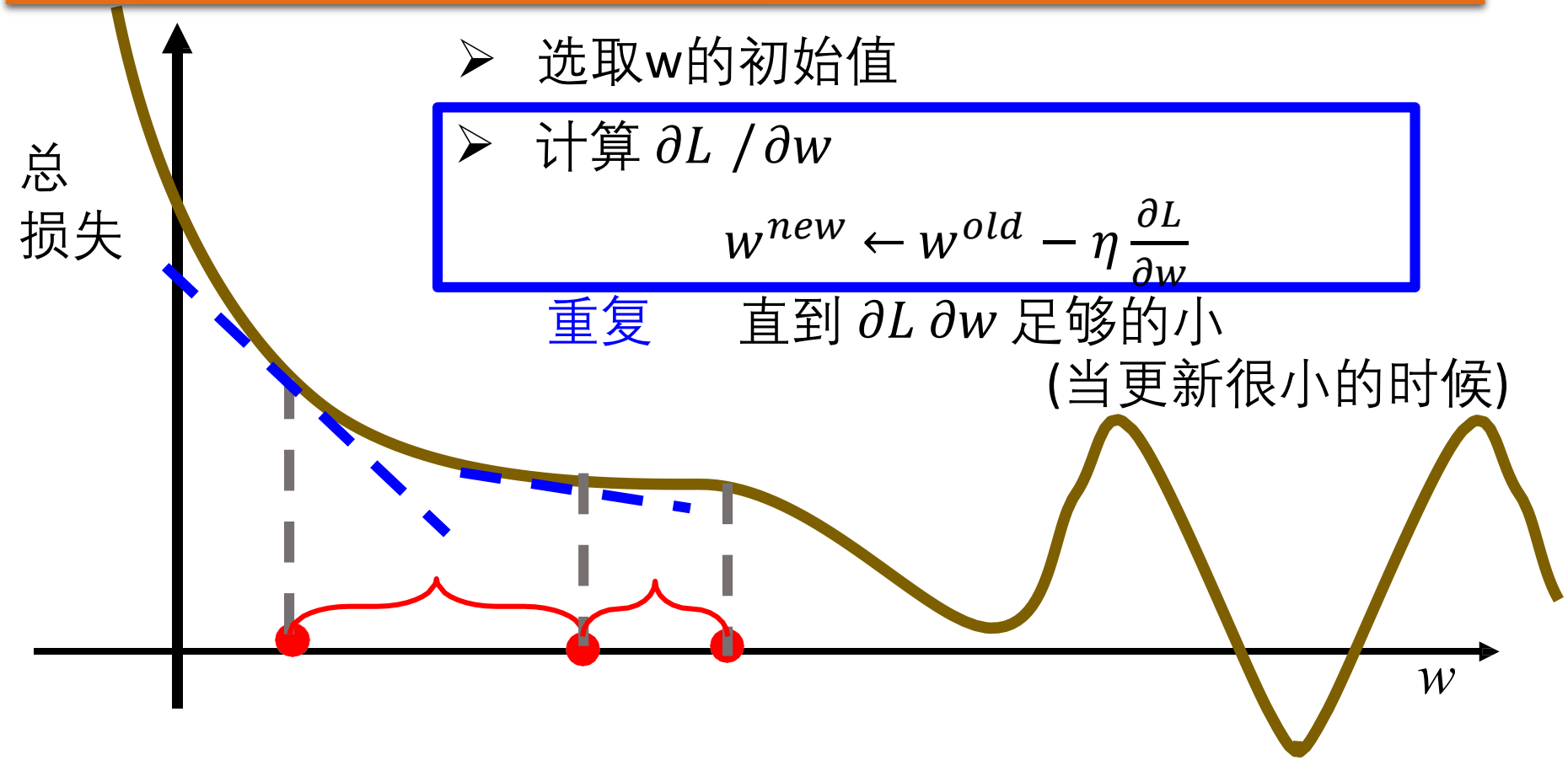
找到使总损失最小的参数$\boldsymbol{\theta}^*$

- 选取w的初始值
- 计算 $\partial L / \partial w$

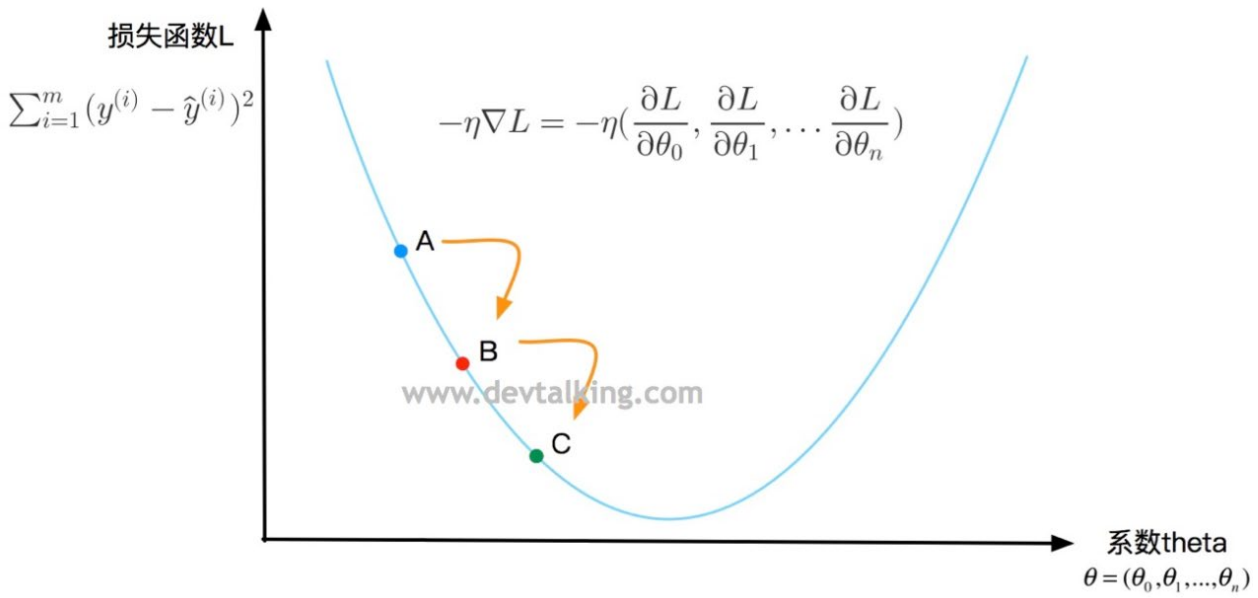
$$w^{new} \leftarrow w^{old} - \eta \frac{\partial L}{\partial w}$$

重复　直到 $\partial L\ \partial w$ 足够的小

(当更新很小的时候)

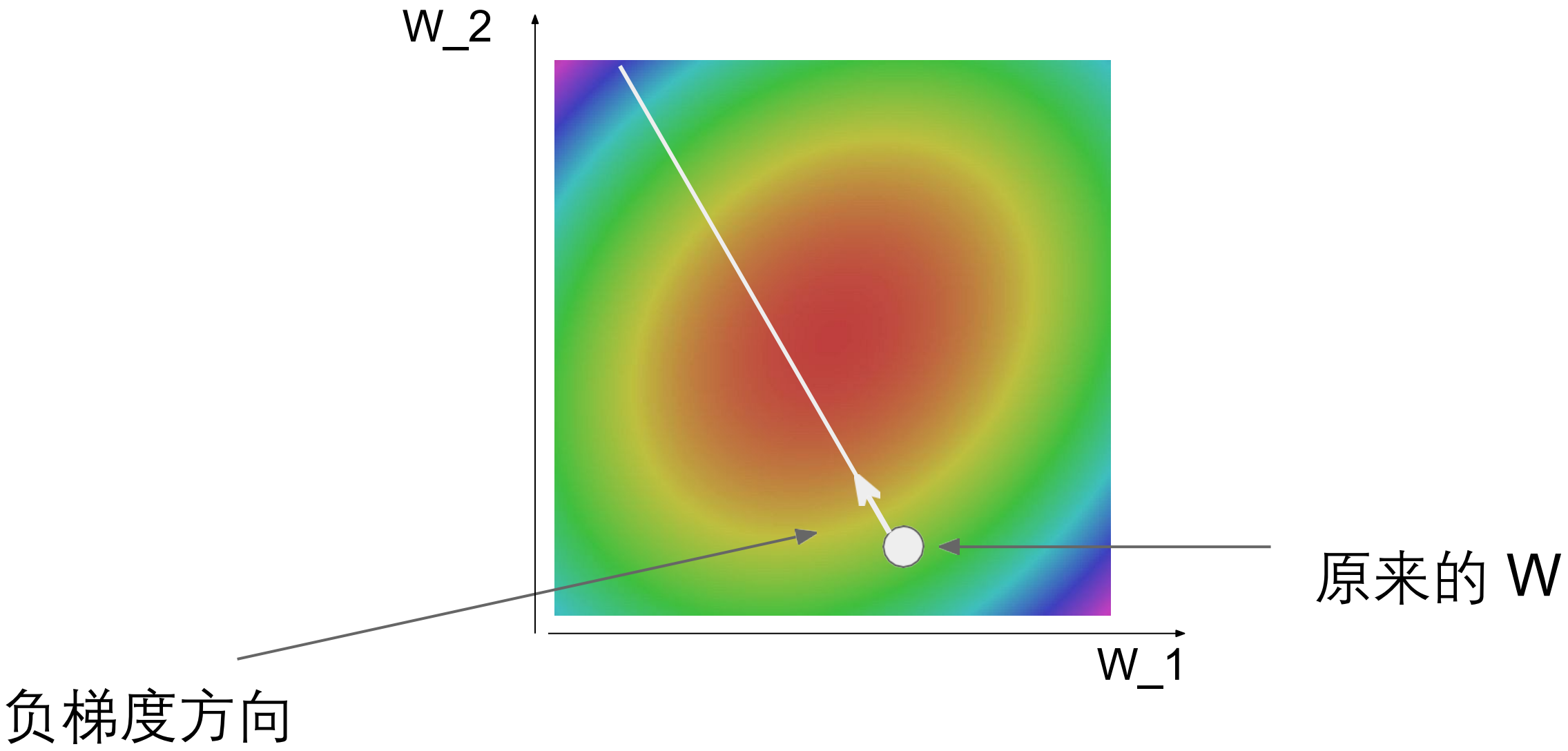


梯度下降




```
# Vanilla Gradient Descent

while True:
  weights_grad = evaluate_gradient(loss_fun, data, weights)
  weights += - step_size * weights_grad # perform parameter update
```


W_2
W_1
原来的 W
负梯度方向

随机梯度下降法 (SGD)

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(x_i, y_i, W) + \lambda R(W)$$

$$\nabla_W L(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

当N很大时，整个计算需要花费大量资源。

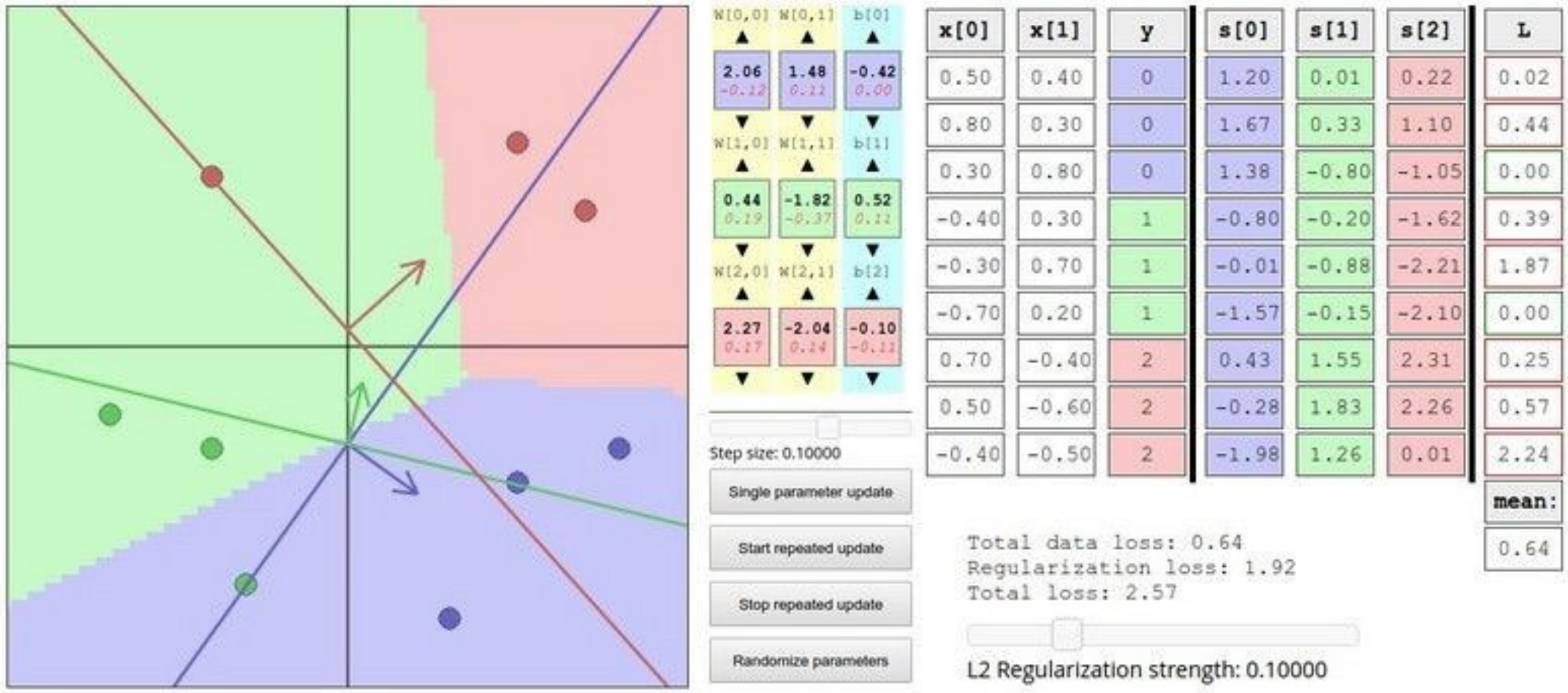
使用小批量示例近似求和

常用 32 / 64 / 128

```
# Vanilla Minibatch Gradient Descent

while True:
  data_batch = sample_training_data(data, 256) # sample 256 examples
  weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
  weights += - step_size * weights_grad # perform parameter update
```

Interactive Web Demo



http://vision.stanford.edu/teaching/cs231n-demos/linear-classify/